

Liberté • Égalité • Fraternité

RÉPUBLIQUE FRANÇAISE



Rapport de Projet

Drone cible

Partie personnelle
BTS CIEL – option IR

BERNARD GOARANT Timéo
2025-2026

1. Périmètre d'activité.....	3
1.1 Mon rôle dans le projet.....	3
1.2 Liste de mes activités individuelles.....	3
1.3 Diagramme de cas d'utilisation de mon périmètre.....	3
2. Environnement de développement.....	4
2.1 Justification des outils et langages choisis.....	4
2.2 Justification des librairies utilisées.....	5
2.3 Identification des contraintes du client.....	5
3. Architecture du projet.....	6
4. Développement – état d'avancement.....	7
4.1 Composants développés.....	7
4.2 Extraits de code.....	9
4.3 Tableaux de tests unitaire.....	10
4.4 Gestion des versions.....	12
5. Exploitation réseau et matériels.....	13
5.1 Présentation des équipements.....	13
5.2 Justification des choix matériels.....	15
5.3 Protocoles et formats de données échangés.....	16
5.4 Cybersécurité.....	20
6. Bilan personnel.....	20
6.1 Ce que j'ai réalisé.....	20
6.2 Difficultés rencontrées et solutions apportées.....	21
6.3 Ce que j'ai appris.....	21
7. Conclusion générale.....	21
Annexes.....	22
[1] Commande git pour voir les commits.....	22
[2] Codes complets.....	22
[3] Diagramme d'architecture globale matérielle et logicielle du système du Drone cible.....	30

1. Périmètre d'activité

1.1 Mon rôle dans le projet

Au sein de l'équipe, mon rôle a été de développer et mettre en œuvre le mode de pilotage manuel du drone. Ce mode constitue la fonctionnalité de base du système : il permet à un opérateur de contrôler le drone à distance via une manette de type gamepad, en utilisant une communication radio longue portée LoRa en point à point. J'ai également été en charge de la supervision en temps réel des données essentielles (télémétrie, GPS) affichées sur l'interface opérateur, ainsi que de l'intégration des conteneurs Docker associés au mode manuel.

1.2 Liste de mes activités individuelles

Dans le cadre de ce projet, mes activités individuelles ont été les suivantes :

- Développement du nœud `bridge_manette_node` : récupération des commandes de la manette via le topic ROS2 `/joy`, encodage dans une trame personnalisée et transmission au drone via le module LoRa.
- Développement du nœud `bridge_robot_node` : réception et décodage des trames côté drone, publication des commandes sur le topic `/cmd_vel`, et retransmission des données capteurs (GPS, IMU, caméra) vers le poste opérateur.
- Développement de l'interface Node-RED : création du dashboard de supervision permettant à l'opérateur de visualiser en temps réel la position GPS, les données IMU et les informations de détection.
- Intégration Docker : mise en place des conteneurs nécessaires au fonctionnement du mode manuel sur la Raspberry Pi 5.
- Tests et mesures LoRa : validation de la portée et de la stabilité du lien radio (RSSI, latence), et analyse du temps de réaction entre commande et action moteur.
- Documentation : rédaction des procédures d'association LoRa, de test d'échange et de bascule de mode.

1.3 Diagramme de cas d'utilisation de mon périmètre

Le diagramme de cas d'utilisation ci-dessous délimite le périmètre fonctionnel des composants dont j'avais la charge, en identifiant l'acteur principal l'opérateur qui pilote le drone.

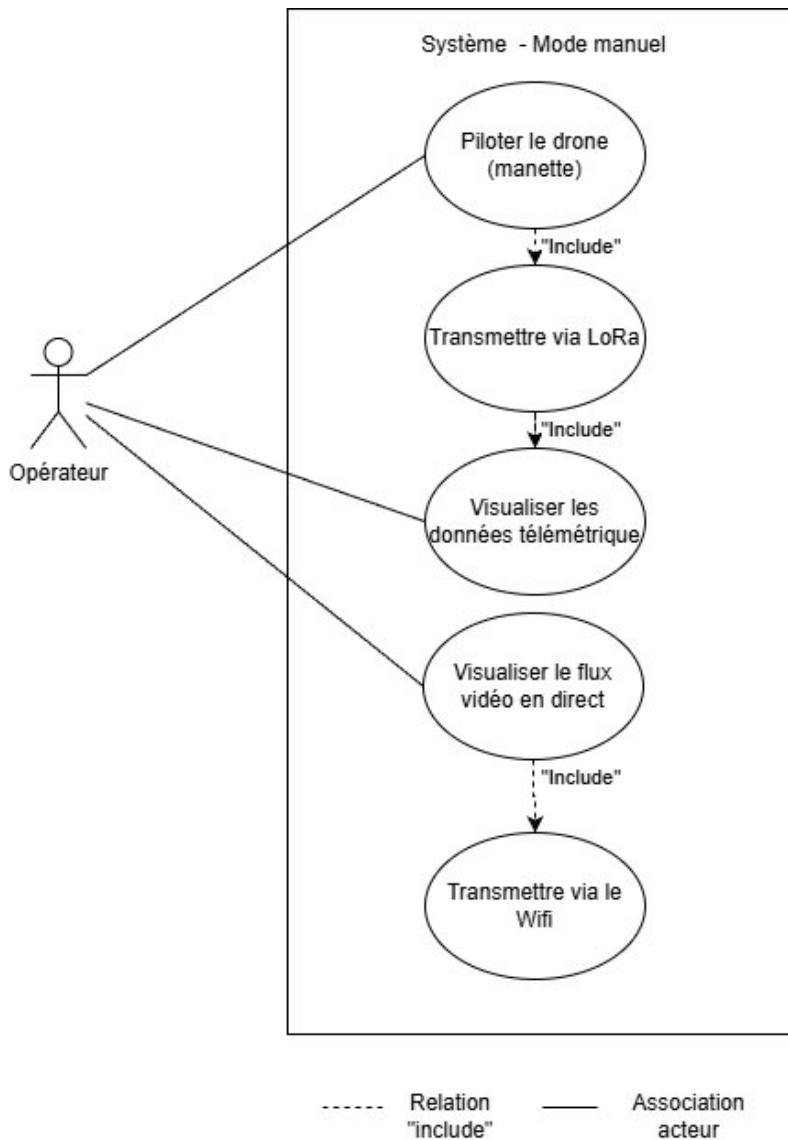


Schéma 1 : Diagramme de cas d'utilisation du mode Manuel

Ce diagramme illustre les interactions entre l'opérateur et le système de pilotage manuel, en mettant en évidence la chaîne fonctionnelle mise en place. L'opérateur agit sur quatre cas d'utilisation : le pilotage du drone via la manette, la transmission des commandes par liaison LoRa, la visualisation des données télémétriques et le suivi du flux vidéo en direct. Ce dernier s'appuie sur une transmission Wi-Fi, tandis que le pilotage inclut systématiquement le passage par la liaison LoRa, garantissant ainsi la communication longue portée avec le drone.

2. Environnement de développement

2.1 Justification des outils et langages choisis

Pour ce projet, ROS2 a été retenu comme framework principal en raison de son architecture distribuée, particulièrement adaptée à l'intégration de multiples capteurs et actionneurs. Sa gestion des messages via topics et services permet de structurer efficacement la communication entre les différents modules du drone.

La Raspberry Pi 5 a été choisie pour sa puissance de calcul suffisante pour gérer simultanément la capture vidéo, le traitement des données capteurs et la communication LoRa en temps réel, tout en restant dans une démarche low-cost cohérente avec les contraintes du projet.

Docker a été adopté pour isoler les environnements d'exécution et garantir une homogénéité de déploiement entre les machines de développement et la carte embarquée. En parallèle, une VM Ubuntu 22.04 avec ROS2 installé nativement est utilisée pour faciliter le développement et les tests dans un environnement stable et contrôlé.

Python a été choisi comme langage principal pour sa lisibilité, la richesse de ses bibliothèques dédiées au traitement des capteurs, et son intégration naturelle avec ROS2, ce qui facilite le prototypage et les tests fonctionnels.

Node-RED a été retenu pour la conception du dashboard de supervision de l'opérateur. Cet outil de programmation visuelle par flux permet de créer rapidement des interfaces graphiques web sans nécessiter de développement front-end complexe. Il communique avec les nœuds ROS2 via le protocole MQTT ou des appels HTTP, récupérant ainsi les données télémétriques — position GPS, données IMU — pour les afficher en temps réel. Son environnement basé sur des blocs interconnectés appelés "flows" permet de modifier l'interface rapidement lors des phases de test, ce qui représente un avantage significatif dans un projet à délais contraints comme celui-ci.

2.2 Justification des librairies utilisées

Plusieurs librairies Python et packages ROS2 sont utilisés dans le projet :

- `rclpy` et `rclpy.node` : fournissent les fonctionnalités de base pour créer des nœuds ROS2, publier et souscrire à des topics, et gérer les services.
- `sensor_msgs.msg import Joy` : utilisé pour la gestion des commandes depuis un joystick ou tout autre interface de contrôle.
- `geometry_msgs.msg import Twist` : permet de gérer les commandes de déplacement du robot ou des modules mobiles.
- `serial` : librairie standard Python utilisée pour la communication avec des modules matériels via des ports série (ex. IMU, GPS).

Ces librairies ont été choisies pour leur compatibilité directe avec ROS2 et la Raspberry Pi, ainsi que pour leur robustesse et leur documentation complète, ce qui assure un développement fiable et modulable.

2.3 Identification des contraintes du client

Plusieurs contraintes techniques ont été définies pour répondre aux besoins du projet.

La première contrainte concerne la portée de communication. Le système doit permettre une communication stable sur une distance minimale de 3 kilomètres. Pour répondre à cette exigence, une communication radio utilisant la technologie LoRa en liaison point à point a été mise en place, grâce à deux antennes dédiées.

Une seconde contrainte concerne l'ergonomie de pilotage pour l'opérateur. Le système doit être facilement contrôlable, avec une interface intuitive. Pour cela, le pilotage est réalisé à l'aide d'une manette de jeu de type DualShock 4, offrant une prise en main familière et précise pour l'utilisateur.

Une autre contrainte importante concerne la qualité du flux vidéo. Le système doit permettre la diffusion d'un flux vidéo en temps réel provenant d'une webcam embarquée, avec une qualité suffisante pour permettre à l'opérateur de visualiser correctement l'environnement.

Enfin, certaines contraintes logicielles ont été imposées :

- L'ensemble du développement logiciel doit être réalisé avec ROS2 afin d'assurer une architecture modulaire et distribuée.
- L'interface utilisateur doit être développée avec Node-RED, qui permet de créer rapidement des interfaces graphiques interactives et de visualiser les données du système.

L'environnement de développement mis en place permet de répondre aux besoins techniques du projet tout en garantissant une architecture flexible et évolutive. L'utilisation de Robot Operating System 2 (ROS2) associée au langage Python facilite la communication entre les différents modules du système et l'intégration des capteurs. Le déploiement sur Raspberry Pi 5, ainsi que l'utilisation de conteneurs Docker et d'un environnement de développement sous Ubuntu 22.04, permet également d'assurer une bonne portabilité et une gestion maîtrisée de l'environnement logiciel.

Les outils et technologies sélectionnés permettent ainsi de répondre aux contraintes du projet, notamment en matière de communication longue portée, d'ergonomie de pilotage et de diffusion du flux vidéo.

Afin de mieux comprendre l'organisation globale du système, la partie suivante présentera l'architecture du projet. Celle-ci décrira la structure logicielle et matérielle du système à travers différents diagrammes, ainsi que l'organisation du réseau et la circulation des données entre les différents composants.

3. Architecture du projet

L'architecture d'un système permet de représenter l'organisation des différents composants matériels et logiciels ainsi que leurs interactions. Dans ce projet, le système s'appuie sur Robot Operating System 2 (ROS2) déployé sur une Raspberry Pi 5, permettant la communication entre les capteurs, les modules de transmission LoRa et l'interface opérateur développée avec Node-RED.

Afin de mieux comprendre l'organisation globale du projet, cette partie présentera l'architecture logicielle de la partie manuel.

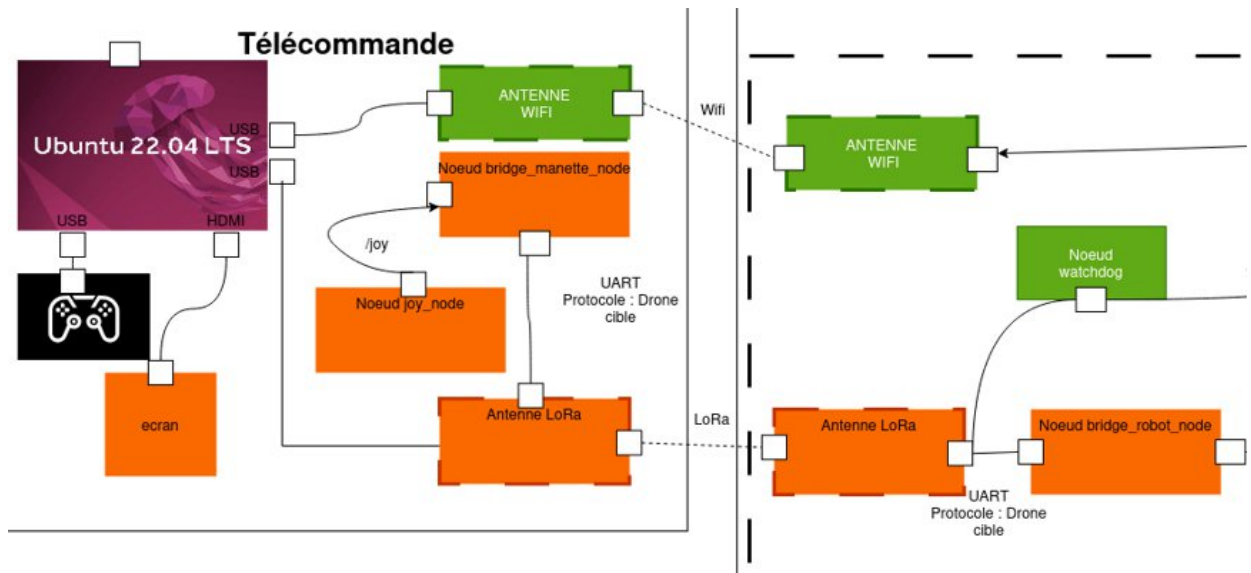


Schéma 2 : [Diagramme d'architecture](#) partie mode manuel du Drone cible

La partie télécommande correspond au poste opérateur et regroupe les éléments permettant de piloter le drone à distance. Elle repose sur un ordinateur exécutant Ubuntu 22.04, connecté à une manette de jeu utilisée pour générer les commandes de pilotage. Le nœud `joy_node` récupère les informations provenant de la manette et les publie sous forme de messages ROS2. Ces données sont ensuite traitées par le nœud `bridge_manette_node`, qui assure l'adaptation et la transmission des commandes vers le système embarqué. La communication avec le drone est réalisée grâce à deux antennes LoRa en point-à-point (P2P), permettant d'assurer une liaison radio longue portée (supérieur à 5km en zone dégagée). L'ensemble de ces nœuds constitue l'interface de contrôle entre l'opérateur et le drone, en convertissant les actions de l'utilisateur en commandes exploitables par le système embarqué.

4. Développement – état d'avancement

4.1 Composants développés

Dans le cadre de ce projet, plusieurs composants logiciels ont été développés afin d'assurer la communication entre le poste de pilotage et le robot, ainsi que l'affichage des informations pour l'opérateur. Ces composants reposent sur l'architecture distribuée de ROS2.

– `bridge_manette_node` :

Le `bridge_manette_node` est exécuté sur la machine virtuelle fonctionnant sous Ubuntu 22.04. Ce nœud a pour rôle de récupérer les données issues de la manette de contrôle et de les transmettre au robot en utilisant un protocole de communication spécifiquement développé dans le cadre du projet.

Le nœud s'abonne au topic `/joy`. Les données sont ensuite encapsulées dans une trame respectant le protocole que nous avons défini pour la communication radio. Cette trame est envoyée au robot via le module de communication LoRa, ce qui permet d'assurer le pilotage du robot à longue distance.

Ce nœud assure également la réception des trames provenant du robot. Les informations contenues dans ces trames (position GPS, données IMU ou informations issues de la détection d'objets) sont décodées puis transmises vers l'interface utilisateur afin d'être affichées.

– bridge_robot_node :

Le `bridge_robot_node` est exécuté sur la Raspberry Pi 5 embarquée sur le robot. Son rôle principal est de servir d'interface entre la communication radio et l'architecture ROS2 du robot.

Ce nœud reçoit les trames de direction du drone envoyées par le poste opérateur et les décode afin d'extraire les informations de commande. Ces données sont ensuite converties en messages ROS2 et publiées sur le topic `/cmd_vel`, permettant ainsi de contrôler les déplacements du robot.

En parallèle, ce nœud collecte plusieurs informations provenant des différents capteurs embarqués, notamment :

- Les données de position du GPS,
- Les données d'orientation issues de l'IMU,
- Les informations provenant du système de détection basé sur la caméra (position et dimension du rectangle de détection généré par l'algorithme de vision Yolo V5).

Ces données sont ensuite encapsulées dans une trame du protocole de communication et transmises vers le poste opérateur afin d'être exploitées par le dashboard Node-Red.

– Interface Node-red :

L'interface utilisateur a été développée à l'aide de Node-Red. Cette interface permet à l'opérateur de visualiser en temps réel les informations transmises par le robot.

Elle affiche notamment :

- La position GPS du robot,
- Les informations issues de l'IMU,
- Les données liées à la détection de bateaux,
- Les informations nécessaires au pilotage du drone.

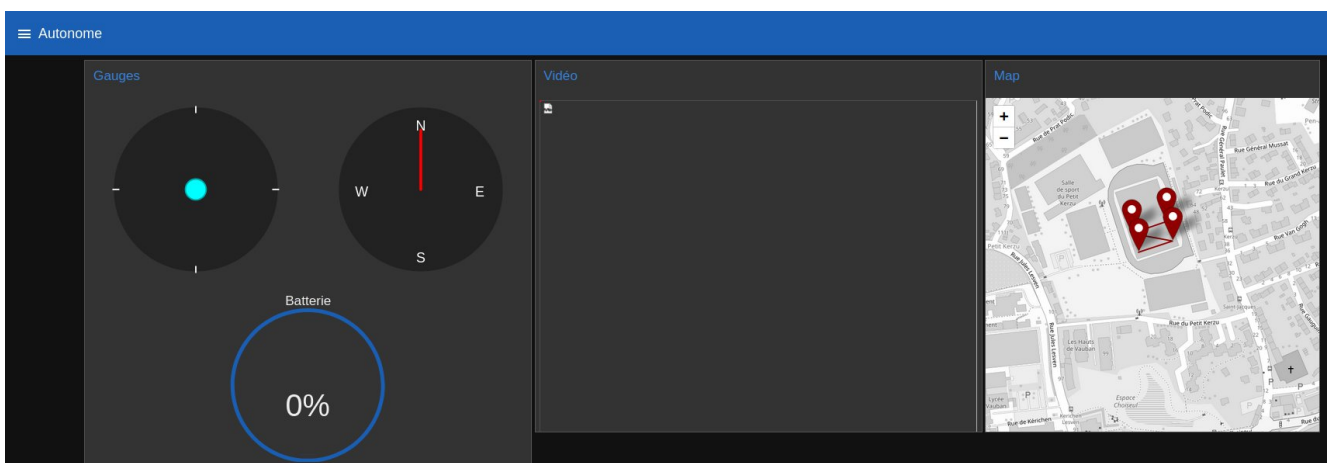


Photo 1 : Capture d'écran de l'interface Node-Red

L'utilisation de Node-RED permet de créer rapidement une interface graphique interactive et facilement modifiable, facilitant ainsi la supervision et le contrôle du robot pendant son utilisation.

4.2 Extraits de code

L'extrait de code suivant correspond au nœud `bridge_manette_node.py`, développé en Python pour l'environnement ROS2. Ce nœud est exécuté sur la machine virtuelle Ubuntu 22.04 et permet de transmettre les commandes de la manette vers le robot la communication LoRa.

4.2.1 Initialisation du nœud et souscription au topic

```
class Bridge_manette(Node):  
    def __init__(self):  
        super().__init__('bridge_manette')  
        self.subscription = self.create_subscription(  
            Joy,  
            '/joy',  
            self.listener_callback,  
            10)  
        try:  
            self.ser = serial.Serial('/dev/ttyUSB0', 9600, timeout=0.1)  
            self.get_logger().info("UART ouvert avec succès")  
        except serial.SerialException as e:  
            self.get_logger().error(f"Erreur ouverture UART: {e}")  
            self.ser = None
```

Dans cette première partie, la classe `Bridge_manette` est définie comme un nœud ROS2. Le nœud souscrit au topic `/joy`, qui contient les données provenant de la manette. À chaque réception d'un message, la fonction `listener_callback` est appelée pour traiter les données.

Le code initialise également une communication série (UART) avec le module LoRa grâce à la bibliothèque `serial`. Cette liaison permet d'envoyer les commandes vers le robot.

4.2.2 Traitement des données de la manette

```
def listener_callback(self, msg):  
    x0, x1 = msg.axes[:2]  
    x0_mapped = int((x0+1)*127.5)  
    x1_mapped = int(x1*10+10)
```

Cette fonction est exécutée à chaque réception d'un message provenant de la manette. Les valeurs des deux axes du joystick de gauche de la manette sont récupérées dans `msg.axes`.

Ces valeurs étant initialement comprises entre -1 et 1, elles sont transformées en valeurs entières adaptées au protocole de communication utilisé pour piloter le robot. La valeur `x0` est alors comprise entre 0 et 255 avec un offset de 127.5 et la valeur `x1` est elle comprise entre 0 et 20 avec un offset de 10.

4.2.3 Construction de la trame et envoi

```

frame = bytearray([
    0x52,      # Start
    0x08,      # Type
    0x02,      # Length
    x0_mapped, # Payload[0]
    x1_mapped  # Payload[1]
])

self.get_logger().info(f"Trame: {[hex(b) for b in frame]}")

if self.ser and self.ser.is_open:
    # conversion en hex
    payload_hex = frame.hex()

    # construction commande AT format envoi pour le LoRa
    cmd = f"AT+SEND=0,{payload_hex},0,0\r\n"

    self.get_logger().info(f"Commande LoRa: {cmd.strip()}")

    # envoi
    self.ser.write(cmd.encode())

```

Dans cette partie du code, une trame de communication personnalisée est construite sous forme de tableau d'octets. Cette trame contient un octet de début, un type de message, la longueur du message et les valeurs correspondant aux commandes du joystick.

La trame est ensuite convertie en format hexadécimal afin d'être intégrée dans une commande AT, utilisée pour communiquer avec le module LoRa via le port série. La commande est finalement envoyée au module radio afin de transmettre les commandes au robot.

4.3 Tableaux de tests unitaire

Afin de valider le bon fonctionnement des développements réalisés, des tests unitaires ont été conduits sur chacun des composants logiciels dont j'avais la charge. Ces tests permettent de vérifier, de manière isolée, que chaque nœud ROS2 se comporte conformément aux exigences du cahier des charges, depuis la lecture des entrées de la manette jusqu'à la transmission des commandes au drone via la liaison LoRa. Les résultats sont consignés dans les tableaux ci-dessous, organisés par composant testé.

4.3.1 Tests du nœud `bridge_manette_node`

Les tests suivants ont pour objectif de valider le fonctionnement du nœud `bridge_manette_node`, depuis la lecture des entrées de la manette jusqu'à la construction de la trame destinée à être envoyée via le module LoRa. Chaque test a été réalisé en observant les topics ROS2 via la commande `ros2 topic echo`, permettant de vérifier en temps réel les données circulant entre les composants.

Cahier de tests — bridge_manette_node						
N°	Objectif	Condition initiale	Action	Résultat attendu	Résultat obtenu	Statut
01	Vérifier la lecture de la manette	Manette connectée, nœud /joy actif	Déplacer le joystick gauche	Publication d'une valeur sur le topic /joy correspondant au mouvement	Valeur correcte publiée sur /joy	✓
02	Vérifier l'abonnement de bridge_manette_node à /joy	bridge_manette_node lancé	Observer les données reçues par le nœud	Données identiques à celles publiées sur /joy	Données identiques confirmées	✓
03	Vérifier la création de la trame LoRa	bridge_manette_node abonné à /joy	Déplacer le joystick et observer la trame générée	Trame encodée correctement formatée à partir des données /joy	Trame créée et correctement formatée	✓

4.3.2 Tests du nœud bridge_robot_node

Les tests suivants valident la chaîne de réception côté drone, depuis la réception de la trame LoRa jusqu'à la publication de la commande sur le topic /cmd_vel et le déplacement effectif du robot. Ces tests ont été réalisés en conditions réelles sur le châssis roulant, conformément aux contraintes de validation définies dans le cahier des charges.

Cahier de tests — bridge_robot_node						
N°	Objectif	Condition initiale	Action	Résultat attendu	Résultat obtenu	Statut
04	Vérifier la réception LoRa côté drone	bridge_robot_node lancé, liaison LoRa active	Envoyer une trame depuis bridge_manette_node	Trame reçue par le module LoRa du drone	Trame reçue correctement	✓
05	Vérifier l'affichage de la commande AT et de la trame	bridge_robot_node en écoute	Envoyer une trame LoRa	Commande AT et contenu de la trame affichés dans les logs	Commande AT et trame affichées correctement dans les logs	✓
06	Vérifier la publication sur /cmd_vel	Trame LoRa reçue et décodée	Observer le topic /cmd_vel après réception	Données de la trame publiées sur /cmd_vel	Données correctement publiées sur /cmd_vel	✓
07	Vérifier le déplacement du robot	/cmd_vel publié, moteurs connectés	Observer le comportement physique du robot	Le robot avance conformément à la commande envoyée	Robot en déplacement confirmé	✓

4.3.3 Tests de portée de la liaison LoRa

Les tests suivants évaluent la qualité et la portée de la liaison radio LoRa entre le poste opérateur et le drone, en fonction de la distance. Chaque test a été réalisé à l'aide de deux modules LoRa connectés via PuTTY : l'un en émission envoyant une commande AT, l'autre en réception affichant la trame reçue avec le RSSI et le SNR associés. Ces deux indicateurs permettent de quantifier la fiabilité de la communication radio à chaque distance testée.

Cahier de tests — Portée et qualité liaison LoRa (PuTTY)

Matériel : 2 modules LoRa via PuTTY — émetteur / récepteur | Commande AT : AT+SEND=0,11,HELLO DRONE

N°	Distance	Commande AT envoyée (émetteur)	Trame reçue (récepteur PuTTY)	RSSI (dBm)	SNR	Trame correcte	Statut
01	10 m	AT+SEND=0,11,HELLO DRONE	+RCV=0,11,HELLO DRONE,-45,12	-45	12	✓	✓
02	30 m	AT+SEND=0,11,HELLO DRONE	+RCV=0,11,HELLO DRONE,-62,10	-62	10	✓	✓
03	50 m	AT+SEND=0,11,HELLO DRONE	+RCV=0,11,HELLO DRONE,-74,9	-74	9	✓	✓
04	100 m	AT+SEND=0,11,HELLO DRONE	+RCV=0,11,HELLO DRONE,-84,7	-84	7	✓	✓
05	200 m	AT+SEND=0,11,HELLO DRONE	+RCV=0,11,HELLO DRONE,-95,5	-95	5	✓	✓

Définition :

RSSI (Received Signal Strength Indicator) : indicateur de la puissance du signal radio reçu, exprimé en dBm. Plus la valeur est proche de 0, meilleur est le signal. En LoRa, un signal est considéré comme fiable au-dessus de -100 dBm et instable en dessous.

SNR (Signal-to-Noise Ratio) : rapport entre la puissance du signal utile et le bruit ambiant. Un SNR élevé indique un signal propre et peu perturbé. En LoRa, un SNR positif est le signe d'une communication de bonne qualité.

4.4 Gestion des versions

Dans le cadre de mes développements, j'ai utilisé Git et le dépôt partagé Drone_cible_2026_repo pour versionner et centraliser mes contributions. Comme le montre la photo 2, le dépôt est organisé en plusieurs dossiers distincts. Mes ajouts se retrouvent principalement dans les dossiers code/ — notamment les nœuds bridge_manette_node et bridge_robot_node ainsi que les flows Node-RED — et dans le dossier docs/ pour la documentation associée.

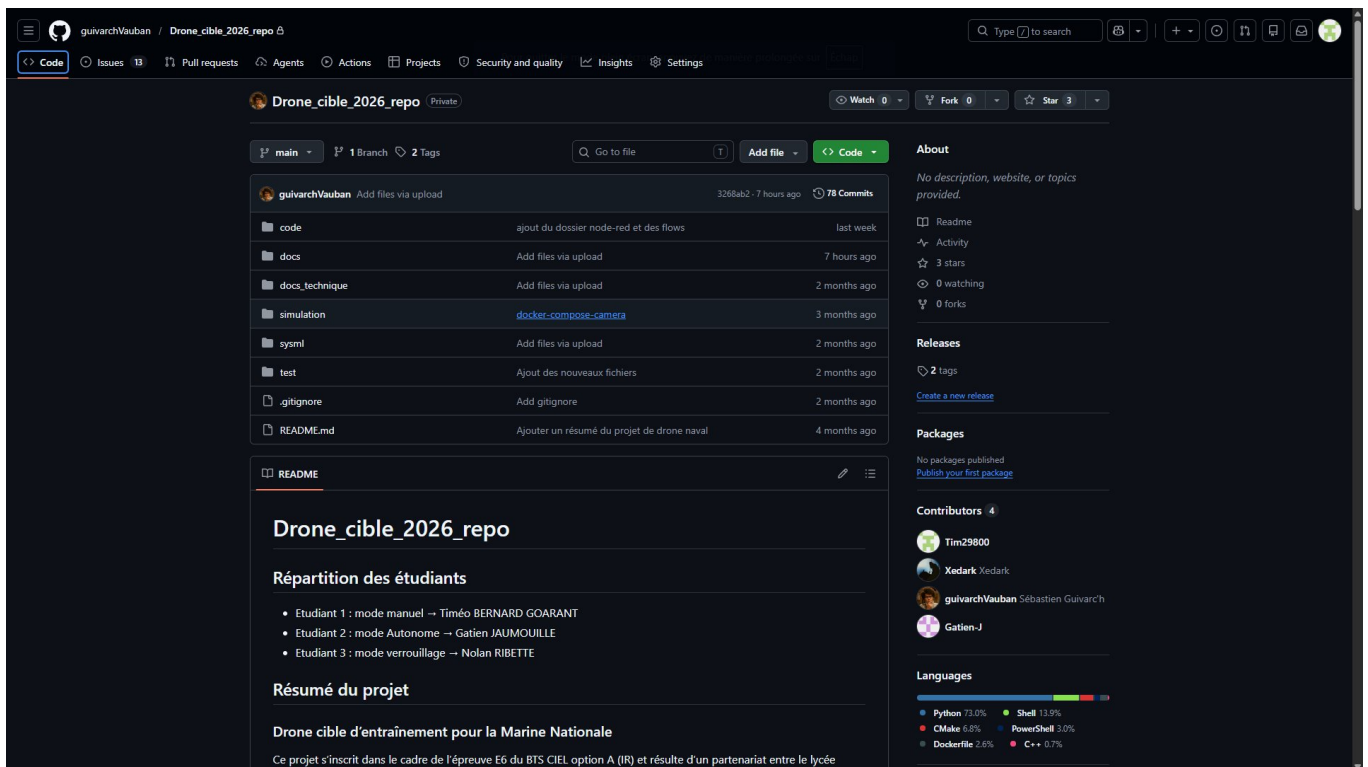


Photo 2 : page d'accueil du dépôt GitHub

L'historique des commits présenté en [annexe 1](#), obtenu via la commande `git log --oneline`, retrace la progression du projet depuis le commit initial jusqu'aux derniers ajouts, parmi lesquels figurent des messages tels que "ajout du dossier node-red et des flows" directement liés à mes activités. Le développement s'étant effectué sur une branche unique `main`, les contributions de chaque membre sont identifiables à travers les messages de commits.

Pour les projets futurs, une pratique de commits plus assidue — un commit par fonctionnalité développée — permettrait un suivi individuel plus précis et faciliterait la revue de code en équipe. Le versionnement mis en place a néanmoins permis d'assurer la traçabilité des développements tout au long du projet et de revenir à un état stable du code en cas d'erreur.

5. Exploitation réseau et matériels

5.1 Présentation des équipements

Dans le cadre de ce projet, plusieurs équipements matériels sont utilisés afin d'assurer le pilotage manuel du robot et la communication entre le poste opérateur et le système embarqué.

La Raspberry Pi 5 constitue l'ordinateur embarqué du robot. Elle exécute les différents nœuds de ROS2 nécessaires au fonctionnement du système.

La communication longue portée entre le poste opérateur et le robot est assurée par une antenne LoRa équipé d'un adaptateur USB LA66. La technologie LoRa permet d'établir une liaison radio fiable sur plusieurs kilomètres tout en consommant peu d'énergie.



Photo 3 : Antenne LoRa + adaptateur USB LA66

Le pilotage du robot est réalisé à l'aide d'une DualShock 3 ou 4. Les informations provenant des joysticks sont récupérées sous forme de messages ROS2 afin de générer les commandes de déplacement du robot.



Photo 4 : DualShock 3

Enfin, le poste opérateur est constitué d'un ordinateur exécutant une machine virtuelle, grâce à Oracle VirtualBox, sous Ubuntu 22.04. À terme, la distribution Ubuntu 22.04 pourra être installée directement sur un PC. Cet environnement permet d'exécuter les nœuds ROS2 nécessaires au contrôle du système ainsi que l'interface développée avec Node-RED.

Afin de garantir la reproductibilité de l'environnement de développement et de faciliter la maintenance du système, l'ensemble des versions des matériels et logiciels utilisés dans le cadre de ce projet sont répertoriées dans le tableau ci-dessous. Ces informations constituent une référence essentielle pour tout déploiement futur du système ou intervention de maintenance.

Tableau des versions — Matériels et logiciels		
Composant	Type	Version
Raspberry Pi 5	Matériel	Modèle 4 Go RAM
Raspberry Pi OS	Système d'exploitation	Debian 12 (Bookworm) 64 bits
ROS2	Framework robotique	Jazzy
Python	Langage	3.12.3
Docker	Conteneurisation	29.3.1
Node-RED	Interface utilisateur	4.1.8
Node.js	Runtime JavaScript	20.20.2
Ubuntu (VM)	Système d'exploitation	22.04 LTS
Oracle VirtualBox	Virtualisation	7.2.0 r170228
Module LoRa LA66	Matériel	LA66 P2P Firmware v1.2.4
Manette DualShock	Matériel	DualShock 3

5.2 Justification des choix matériels

Le choix des équipements utilisés dans ce projet repose sur plusieurs critères, notamment les performances, la simplicité d'intégration et l'adaptation aux besoins du système.

La communication entre le poste opérateur et le robot repose sur la technologie LoRa. Ce choix s'explique principalement par sa grande portée de communication en champ libre, permettant d'atteindre plusieurs kilomètres tout en restant fiable. De plus, cette technologie permet d'envoyer des données personnalisées, ce qui facilite l'intégration de notre protocole adapté aux besoins du projet.

Le système embarqué utilise une Raspberry Pi 5. Cette carte a été choisie pour sa capacité de calcul élevée, sa mémoire vive suffisante et la possibilité d'exécuter une distribution Linux, ce qui permet de faire fonctionner facilement ROS2 et les différents services nécessaires au projet.

Le pilotage du robot est réalisé à l'aide d'une DualShock 3. Cette manette offre une ergonomie adaptée à une utilisation prolongée, ce qui est important pour l'opérateur lors des phases de pilotage. Les manettes de jeu étant conçues pour les joueurs qui les utilisent pendant de longues périodes, elles offrent une prise en main confortable et des commandes précises.

Enfin, l'interface utilisateur a été développée avec Node-RED. Plusieurs outils comme RViz et rqt ont été testés au début du projet, mais Node-RED s'est révélé plus adapté aux besoins du projet. Il permet de créer rapidement une interface graphique complète et facilement modifiable. De

plus, cet outil ayant déjà été utilisé lors de travaux pratiques précédents, sa prise en main a été rapide, ce qui a facilité le développement de l'interface.

5.3 Protocoles et formats de données échangés

Cette section décrit les protocoles et formats de données mis en œuvre pour assurer la communication entre le poste opérateur et le drone. L'ensemble des échanges repose sur une liaison radio LoRa point à point (P2P), au sein de laquelle chaque information est encapsulée dans une trame structurée de format fixe. Sont présentés ci-après le format détaillé de ces trames ainsi que les résultats d'analyse réseau obtenus lors des tests.

5.3.1 Trames de communication LoRa

Le protocole de communication LoRa utilisé dans ce projet repose sur un format de trame personnalisé, conçu pour être léger et adapté aux contraintes de bande passante du module LA66. Chaque trame est composée d'un octet de début fixe (0x52), d'un octet de type identifiant la nature de la donnée, d'un octet indiquant la longueur du payload, des données utiles, et d'un CRC de contrôle d'intégrité. Les tableaux ci-dessous présentent l'ensemble des trames définies, réparties selon leur sens de communication.

Protocole de communication LoRa — Vue générale des trames

ROBOT → MANETTE (données télémétriques)			
TYPE	Nom	Données (Payload)	Description
0x01	GPS	Lat, Long (Float)	Position GPS du drone
0x02	IMU	Cap, Roulis, Tangage (Float)	Orientation et rotation du drone
0x03	Bateau	Détection, X, Y, H, L (Float)	Données de détection visuelle de la cible
0x04	Batterie	Pourcentage (%)	Niveau de batterie du drone
0x05	Acq. Mode	0 / 1 / 2 / 3	Mode d'acquisition actif

MANETTE → ROBOT (commandes opérateur)			
TYPE	Nom	Données (Payload)	Description
0x06	Mode	0=Manuel, 1=Auto, 2=Verrou, 3=SafeMode	Sélection du mode de fonctionnement
0x07	Watchdog	Valeur (int 0–255)	Signal de vie de la télécommande
0x08	/Joy	Linéaire (int), Angulaire (int)	Commandes de déplacement manette

Structure générale d'une trame				
Start (1 octet)	Type (1 octet)	Data Length (1 octet)	Payload (variable)	CRC (1 octet)
0x52	0xXX	N octets	Données utiles	Contrôle d'intégrité

Ce tableau présente la vue d'ensemble du protocole, organisée en deux flux distincts. Le flux Robot → Manette regroupe les données télémétriques remontées par le drone vers l'opérateur : position GPS, orientation IMU, détection visuelle et mode actif. Le flux Manette → Robot regroupe

quant à lui les commandes émises par l'opérateur : sélection du mode de fonctionnement, signal de vie Watchdog et commandes de déplacement issues de la manette via le topic /joy.

Détails des trames — Format octet par octet										
Trame	Start	Type	Length	Payload[0]	Payload[1]	Payload[2]	Payload[3]	Payload[4]	Payload[5]	CRC
GPS	0x52	0x01	0x06	LAT	LAT << 8	LAT << 16	LONG	LONG << 8	LONG << 16	0xXX
/Joy	0x52	0x08	0x02	Linéaire [0;20] (offset +10)	Angulaire [0;255] (offset +127)					0xXX
IMU boussole	0x52	0x02	0x05	int [0;9]	int [0;9]	virgule '.'	int [0;9]	int [0;9]		0xXX
Watchdog	0x52	0x07	0x01	Valeur [0;255]						0xXX
Mode	0x52	0x06	0x01	0/1/2/3						0xXX

Notes d'encodage	
Linéaire (/Joy)	Valeur entre -10 et +10 → encodée entre 0 et 20 avec un offset de +10 (ex : 0 → 10, -5 → 5, +5 → 15)
Angulaire (/Joy)	Valeur entre -180 et +180 → encodée entre 0 et 255 avec un offset de +127
GPS	Latitude et longitude encodées sur 3 octets chacune par décalage de bits (<<8, <<16)
IMU boussole	Cap encodé chiffre par chiffre en ASCII (ex : 123.45 → '1','2','3','.', '4','5')

Celui-ci détaille le format octet par octet de chaque trame. On notera en particulier les conventions d'encodage utilisées pour optimiser la taille des données transmises : les valeurs de joystick sont converties en entiers non signés avec un offset afin d'éviter la transmission de nombres négatifs (linéaire : offset +10, angulaire : offset +127), et les coordonnées GPS sont encodées sur 3 octets chacune par décalage de bits. Ces choix d'encodage permettent de minimiser la taille des trames et donc la latence de transmission sur la liaison LoRa.

5.3.2 Résultat d'analyse de trame au PicoScope

Afin de valider le bon fonctionnement de la communication série entre la machine virtuelle et le module LoRa LA66, une analyse des trames a été réalisée à l'aide d'un oscilloscope numérique PicoScope, branché sur la broche RX du module LoRa et le GND. Le décodeur UART intégré au logiciel PicoScope a permis de décoder en temps réel les caractères transmis sur la liaison série et d'en analyser le contenu et le timing.

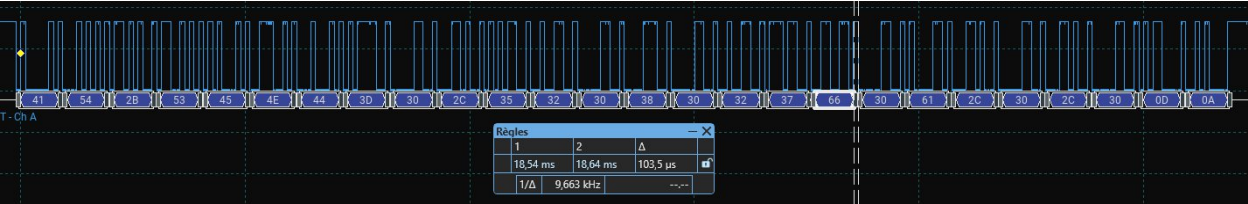


Photo 5 : Mesure de la durée d'un bit sur la liaison UART — PicoScope

La photo 5 présente la mesure de la durée d'un bit sur la liaison série entre la machine virtuelle et le module LoRa LA66. La mesure au PicoScope indique une fréquence de 9,663 kHz, soit une période de 103,5 μs par bit. Cette valeur est cohérente avec une configuration à 9600 bauds, pour laquelle la durée théorique d'un bit est de $1/9600 \approx 104,2 \mu s$. L'écart mesuré est inférieur à 1%, ce qui confirme que la liaison UART est correctement configurée à 9600 bauds sur les deux extrémités de la communication.

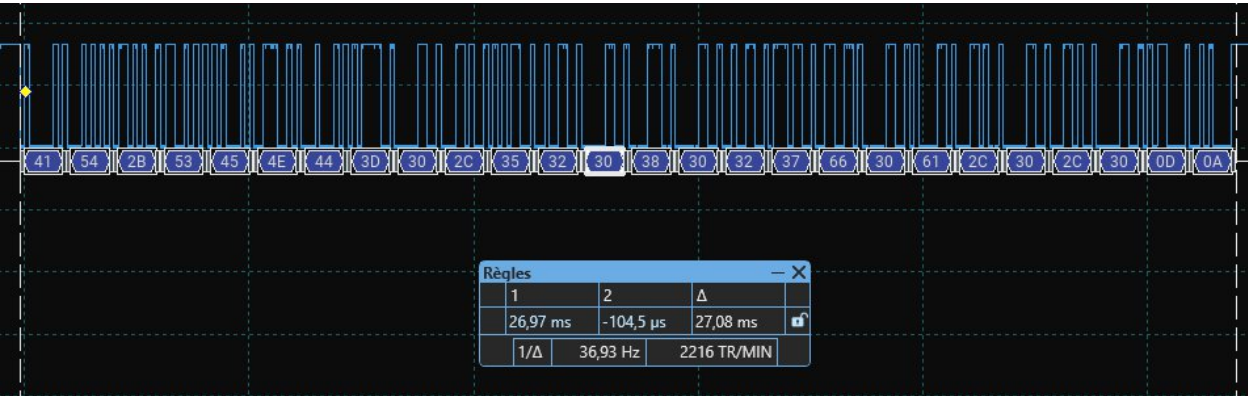


Photo 6 : Mesure de la durée de transmission d'une trame LoRa complète

La photo 6 présente la durée totale de transmission d'une trame complète sur la liaison série. La mesure indique une durée de 27,08 ms par trame, ce qui correspond à une fréquence maximale théorique de $1000 / 27,08 \approx 36,9$ trames par seconde. En pratique cependant, le module LoRa LA66 nécessite un temps de traitement entre chaque envoi, et la liaison radio elle-même introduit une latence supplémentaire. En appliquant une marge de sécurité de 50%, il est raisonnable de limiter la fréquence d'envoi à environ 18 trames par seconde pour éviter toute saturation de l'antenne. À titre de comparaison, la fréquence de publication actuelle des topics ROS2 est volontairement réduite en dessous de ce seuil, conformément aux ajustements réalisés suite aux problèmes de saturation rencontrés lors des tests.

Analyse PicoScope — Décodage UART de la trame LoRa (AT+SEND)					
Liaison analysée : RX du module LoRa LA66 — Décodeur UART PicoScope — 9600 bauds, 8N1					
N° paquet	Caractère	Début (ms)	Fin (ms)	Durée (μs)	Rôle dans la trame
1	A	-0.104	0.938	1042	Commande AT — 'A'
2	T	0.938	1.980	1042	Commande AT — 'T'
3	+	1.979	3.021	1042	Commande AT — '+'
4	S	3.020	4.062	1042	Commande AT — 'S'
5	E	4.062	5.104	1042	Commande AT — 'E'

6	N	5.103	6.145	1042	Commande AT — 'N'
7	D	6.145	7.187	1042	Commande AT — 'D'
8	=	7.186	8.228	1042	Séparateur
9	0	8.227	9.269	1042	Adresse destinataire
10	,	9.269	10.311	1042	Séparateur
11	5	10.310	11.352	1042	Longueur payload — '5'
12	2	11.352	12.394	1042	Longueur payload — '2'
13	0	12.393	13.435	1042	Payload — 0x52 (Start)
14	8	13.434	14.476	1042	Payload — 0x08 (Type /Joy)
15	0	14.476	15.518	1042	Payload — 0x02 (Length)
16	2	15.517	16.559	1042	Payload — 0x02
17	7	16.559	17.601	1042	Payload — 0x7F (Linéaire = 127 → neutre)
18	f	17.600	18.642	1042	Payload — 0x7F suite
19	0	18.641	19.683	1042	Payload — 0x0A (Angulaire = 10)
20	a	19.683	20.725	1042	Payload — 0x0A suite
21	,	20.724	21.766	1042	Séparateur
22	0	21.766	22.808	1042	Paramètre optionnel
23	,	22.807	23.849	1042	Séparateur
24	0	23.848	24.890	1042	Paramètre optionnel
25	CR	24.890	25.932	1042	Fin de trame — Carriage Return
26	LF	25.931	26.973	1042	Fin de trame — Line Feed

● Fond vert = octets du payload de la trame LoRa (données utiles)

Récapitulatif de la trame décodée	
Commande AT complète	AT+SEND=0,5,20827f0a,0,0\r\n
Start byte	0x52
Type	0x08 → commande /Joy (déplacement)
Length	0x02 → 2 octets de payload
Payload[0] — Linéaire	0x7F = 127 → offset -10 = valeur neutre (joystick centré)
Payload[1] — Angulaire	0x0A = 10 → offset -127 = -117 (légère rotation)
Fin de trame	CR + LF (0x0D 0x0A)
Durée totale mesurée	≈ 27,08 ms (36,93 trames/sec théorique)

Le tableau ci-dessus présente le décodage UART octet par octet de la commande AT envoyée au module LoRa, capturée par le PicoScope. On retrouve la commande AT+SEND=0,5,20827f0a,0,0\r\n, qui correspond exactement à la trame générée par le bridge_manette_node visible dans les logs ROS2 dont voici un extrait.

```
[INFO] [1779356046.644790480] [bridge_manette]: Trame: ['0x52', '0x8', '0x2', '0x7f', '0xa']  
[INFO] [1779356046.646056033] [bridge_manette]: Commande LoRa:  
AT+SEND=0,5208027f0a,0,0
```

Le payload décodé contient bien les 5 octets attendus : le start byte 0x52, le type 0x08 correspondant à une commande /joy, la longueur 0x02, puis les deux octets de données — 0x7F pour la valeur linéaire (joystick centré, valeur neutre) et 0x0A pour la valeur angulaire. Cette analyse confirme que l'encodage des trames dans le nœud ROS2 est correct et que les données arrivent intactes au module LoRa.

5.4 Cybersécurité

Plusieurs mesures ont été mises en place afin d'améliorer la sécurité du système. L'accès à la Raspberry Pi 5 se fait à distance via le protocole SSH, en utilisant pour les développeurs une clé SSH, ce qui permet d'éviter l'utilisation de mots de passe pour l'authentification et est donc plus rapide. La Raspberry Pi ainsi que la machine virtuelle sous Ubuntu 22.04 sont également protégées par des mots de passe afin de limiter les accès non autorisés.

Le réseau Wi-Fi utilisé pour les communications locales est sécurisé avec le protocole WPA2-Personal, garantissant un chiffrement des échanges sur le réseau sans fil.

La communication radio entre les deux modules utilisant la technologie LoRa est configurée en mode point à point privé, ce qui limite les possibilités d'interception ou d'accès par des équipements extérieurs.

Enfin, l'utilisation de Docker sur la Raspberry Pi permet d'isoler les différents programmes selon le mode de fonctionnement du robot. Cette isolation des services améliore la sécurité globale du système en limitant les interactions directes entre les différentes applications.

L'ensemble des équipements et protocoles présentés dans cette partie constituent l'infrastructure matérielle et réseau sur laquelle repose le système de pilotage manuel du drone. Les choix effectués — LoRa pour la communication longue portée, Raspberry Pi 5 pour le calcul embarqué, DualShock pour l'ergonomie de pilotage — répondent directement aux exigences du cahier des charges, notamment en matière de portée, de fiabilité et de contrainte budgétaire. Les mesures de cybersécurité mises en place garantissent par ailleurs un accès maîtrisé au système, aussi bien en développement qu'en conditions d'utilisation réelle.

6. Bilan personnel

6.1 Ce que j'ai réalisé

Au terme de ce projet, le mode de pilotage manuel du drone est fonctionnel dans son ensemble. Les nœuds bridge_manette_node et bridge_robot_node ont été développés et testés avec succès, la chaîne de communication complète — de la manette jusqu'aux moteurs du drone via la liaison LoRa — a été validée, et le dashboard Node-RED permet une supervision en temps réel des données télémétriques. À ce stade de la rédaction du rapport, les développements restants concernent principalement l'intégration finale des différents programmes de test en un Rapport individuel

unique nœud cohérent et complet pour chaque bridge, ainsi que l'amélioration de l'interface Node-RED, aussi bien sur le plan esthétique que fonctionnel, avec l'ajout de nouvelles fonctionnalités pratiques pour l'opérateur.

6.2 Difficultés rencontrées et solutions apportées

La première difficulté majeure a été la migration de ROS2 Humble vers ROS2 Jazzy, rendue nécessaire pour assurer la compatibilité avec le logiciel de simulation utilisé par l'équipe. Ce changement de version a impliqué une adaptation de l'environnement de développement et une vérification de la compatibilité des dépendances.

La seconde difficulté, plus technique, est liée aux limites internes du module LoRa. Ce protocole radio n'est pas conçu pour transmettre un volume important de données en continu, or le système nécessite l'envoi simultané de plusieurs types de données télémétriques. Une saturation de l'antenne provoquait des instabilités dans la communication. La solution mise en place a été de réduire la fréquence de publication des topics ROS2 afin de ne pas surcharger la liaison radio. Un travail d'optimisation reste cependant à mener pour trouver le meilleur compromis entre la capacité de transmission de l'antenne et la fiabilité de la visualisation des données en temps réel.

6.3 Ce que j'ai appris

Ce projet a été une source d'apprentissage importante, aussi bien sur le plan technique qu'humain. D'un point de vue technique, j'ai découvert ROS2 et Docker, deux outils que je ne connaissais pas avant ce projet et qui sont aujourd'hui largement utilisés dans le domaine de la robotique et du développement logiciel. J'ai également approfondi mon utilisation de Node-RED, que je connaissais déjà mais que je n'avais jamais utilisé à cette échelle pour concevoir une interface de supervision aussi complète. Enfin, ce projet m'a permis de renforcer mes compétences en Python, langage que je maîtrisais déjà et que j'ai pu approfondir davantage, ce qui me sera certainement utile dans ma future carrière.

Sur le plan humain, travailler en équipe sur un projet de cette envergure, en lien avec un partenaire institutionnel comme la Marine Nationale, m'a appris à m'organiser, à communiquer régulièrement avec les autres membres de l'équipe et à m'adapter rapidement aux imprévus techniques.

7. Conclusion générale

Ce projet de drone naval de surface m'a permis de mettre en pratique et d'approfondir des compétences techniques variées dans un contexte professionnel concret et exigeant. De la conception des nœuds ROS2 à l'intégration de la liaison LoRa, en passant par le développement de l'interface de supervision Node-RED, chaque étape a contribué à forger une vision globale du développement d'un système embarqué communicant. Mené en partenariat avec la Marine Nationale, ce projet restera une expérience marquante de ma formation, alliant rigueur technique, travail collaboratif et adaptation aux contraintes du monde réel.

Annexes

[1] Commande git pour voir les commits

```

6c7fbc2 Rename yolo_node.py to yolo_node_affichage.py
c9cc89f Create README.md
d9229b8 yolo_node_python
12ef66d Create yolo_node.py
00ce787 Setup_python_node_yolo
cdcbaab Setup_v1_node_yolov5
66cd4d6 Rename simulation/simulation_traitement_camera/yolov5_script.py to simulation/simulation_traitement_camera/yolov5_node/setup.py
3f72354 Rename Test.md to yolov5_script.py
b71c7f9 Create Test.md
9dc2342 Update notes for project review
48a1a79 Add project review notes and suggestions
2010661 Add files via upload
cc2ed65 Ajouter un résumé du projet de drone naval
8b16ace Add section on technologies used in the project
9182a22 Merge branch 'main' of https://github.com/guivarchVauban/Drone_cible_2026_repo
8028c65 Fix indentation for visual detection mode description in README.md
b70d1c7 Fix description for autonomous mode in README.md
530d17e Add description for visual detection mode in README.md
7a1f3b5 Merge branch 'main' of https://github.com/guivarchVauban/Drone_cible_2026_repo
32fd035 Update README
195abcc Update README with visual detection mode details
bfa7228 Fix description for autonomous mode in README.md
4039bea Fix links and formatting in README.md
b08c574 Update README with project organization details
578bad7 Add README for simulation project
45cdd6a Add project technical documentation header
466d172 Create README.md
f528eb4 Delete code/Timéo/d
e25e738 Create d
90f9f9e Add project description to README
873fd1 Create README.md
7d2e4af Add explanations for drone operation modes
5131a65 Update student distribution in README.md
23e84fe add test gaten
eacc232 Add section for student distribution
d2d223b Document Google Drive organization in README
343a115 Document Google Drive organization in README
32969ff test vscode github
97574c9 Ajout de la documentation SysML originale
adbb722 Add files via upload
9e37779 Create readme.md
adf669b Add files via upload
70f7c5a Create readme.md
7888ac1 Initial commit

```

Annexe 1 : résultat de la commande `git Log --oneLine`

[2] Codes complets

[2.1] bridge_manette – réception coordonnées rectangle

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Joy
from std_msgs.msg import String
import serial
import json

class Bridge_manette(Node):
    def __init__(self):
        super().__init__('bridge_manette')

```

```

# Subscription /joy
self.subscription = self.create_subscription(
    Joy,
    '/joy',
    self.listener_callback,
    10)

# Publisher /rectangle
self.publisher_rectangle = self.create_publisher(String, '/rectangle',
10)

try:
    self.ser = serial.Serial('/dev/ttyUSB0', 9600, timeout=0.1)
    self.get_logger().info("UART ouvert avec succès")
except serial.SerialException as e:
    self.get_logger().error(f"Erreur ouverture UART: {e}")
    self.ser = None

# Timer lecture série
self.timer = self.create_timer(0.01, self.read_serial)

def listener_callback(self, msg):
    x0, x1 = msg.axes[:2]
    x0_mapped = int((x0+1)*127.5)
    x1_mapped = int(x1*10+10)
    frame = bytearray([
        0x52,
        0x08,
        0x02,
        x0_mapped,
        x1_mapped
    ])
    self.get_logger().info(f"Trame: {[hex(b) for b in frame]}")
    if self.ser and self.ser.is_open:
        payload_hex = frame.hex()
        cmd = f"AT+SEND=0,{payload_hex},0,0\r\n"
        self.get_logger().info(f"Commande LoRa: {cmd.strip()}")
        self.ser.write(cmd.encode())

def read_serial(self):
    if self.ser is None:
        return

    try:
        if self.ser.in_waiting == 0:
            return

        line = self.ser.readline().decode("utf-8", errors="ignore").strip()

```



```
        if not line:
            return

        if "Data: (HEX:)" not in line:
            return

        self.get_logger().info(f"Reçu brut : {line}")

        hex_part = line.split("HEX:")[1].strip()
        hex_values = hex_part.split()
        trame = bytes([int(h, 16) for h in hex_values])

        self.get_logger().info(f"Trame reçue : {[hex(b) for b in trame]}")

        if len(trame) < 3:
            return

        start = trame[0]
        frame_type = trame[1]
        nb = trame[2]

        if start != 0x52 or frame_type != 0x03:
            self.get_logger().warn(f"Trame inconnue : start={hex(start)}\ntype={hex(frame_type)}")
            return

        self.get_logger().info(f"Nombre de détections : {nb}")

        detections = []

        for i in range(nb):
            offset = 3 + i * 8
            if offset + 8 > len(trame):
                break

            x = trame[offset] | (trame[offset+1] << 8)
            y = trame[offset+2] | (trame[offset+3] << 8)
            w = trame[offset+4] | (trame[offset+5] << 8)
            h = trame[offset+6] | (trame[offset+7] << 8)

            self.get_logger().info(
                f" Détection {i+1} -> x={x} y={y} width={w} height={h}"
            )
```

```

        detections.append({
            "x": x,
            "y": y,
            "width": w,
            "height": h
        })

    # Publication sur /rectangle
    msg = String()
    msg.data = json.dumps(detections)
    self.publisher_rectangle.publish(msg)
    self.get_logger().info(f"Publié sur /rectangle : {msg.data}")

except Exception as e:
    self.get_logger().warn(f"Erreur lecture série : {e}")

def destroy_node(self):
    if self.ser is not None and self.ser.is_open:
        self.ser.close()
        self.get_logger().info("UART fermé")
    super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    bridge_manette = Bridge_manette()
    try:
        rclpy.spin(bridge_manette)
    except KeyboardInterrupt:
        pass
    bridge_manette.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

[2.2] bridge_drone – direction du drone

```

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
import serial

class BridgeDrone(Node):

    def __init__(self):

        super().__init__('bridge_drone')

```

```
# Paramètres
self.declare_parameter("port", "/dev/ttyUSB0")
self.declare_parameter("baudrate", 9600)
self.declare_parameter("max_linear_speed", 1.0)
self.declare_parameter("max_angular_speed", 1.0)

port = self.get_parameter("port").get_parameter_value().string_value
baud = self.get_parameter("baudrate").get_parameter_value().integer_value

self.max_linear = self.get_parameter("max_linear_speed").value
self.max_angular = self.get_parameter("max_angular_speed").value

# Publisher cmd_vel
self.publisher_ = self.create_publisher(Twist, "/cmd_vel", 10)

# UART
try:
    self.ser = serial.Serial(port, baud, timeout=0.1)
    self.get_logger().info(f"UART ouvert : {port} @ {baud}")
except serial.SerialException as e:
    self.get_logger().error(f"Erreur ouverture UART : {e}")
    self.ser = None

# Timer lecture
self.timer = self.create_timer(0.01, self.read_serial)

def parse_frame(self, line):

    if "Data: (HEX:" not in line:
        return None

    try:

        hex_part = line.split("HEX:")[1]
        hex_part = hex_part.replace('"', "").strip()

        hex_values = hex_part.split()

        if len(hex_values) < 5:
            return None

        start = int(hex_values[0], 16)
        frame_type = int(hex_values[1], 16)
        length = int(hex_values[2], 16)
```

```
        if start != 0x52 or frame_type != 0x08 or length != 0x02:
            return None

        x0_mapped = int(hex_values[3], 16)
        x1_mapped = int(hex_values[4], 16)

        # Conversion inverse
        x0 = (x0_mapped / 127.5) - 1
        x1 = (x1_mapped - 10) / 10

        return x0, x1

    except Exception as e:
        self.get_logger().warn(f"Erreur parsing : {e}")
        return None

    def publish_cmd_vel(self, x0, x1):

        # Seuil pour angular.z (rotation)
        if abs(x0) < 0.02: # Si la valeur de x0 est proche de zéro, on la force
à zéro
            x0 = 0.0

        # Seuil pour linear.x (avancer/reculer)
        if abs(x1) < 0.02: # Si la valeur de x1 est proche de zéro, on la force
à zéro
            x1 = 0.0

        msg = Twist()

        # Limite la vitesse linéaire et angulaire
        msg.linear.x = x1 * self.max_linear
        msg.angular.z = x0 * self.max_angular

        # Ajout du seuil pour angular.z (rotation)
        if abs(msg.angular.z) < 0.001: # Si angular.z est proche de zéro, le
forcer à zéro
            msg.angular.z = 0.0

        # Publier le message
        self.publisher_.publish(msg)

        # Log des valeurs envoyées
        self.get_logger().info(
```

```
        f"cmd_vel -> linear.x={msg.linear.x:.2f}\nangular.z={msg.angular.z:.2f}"\n    )\n\n    def read_serial(self):\n\n        if self.ser is None:\n            return\n\n        try:\n\n            if self.ser.in_waiting == 0:\n                return\n\n            line = self.ser.readline().decode("utf-8", errors="ignore").strip()\n\n            result = self.parse_frame(line)\n\n            if result is None:\n                return\n\n            x0, x1 = result\n\n            self.publish_cmd_vel(x0, x1)\n\n        except serial.SerialException as e:\n            self.get_logger().error(f"Erreur lecture UART : {e}")\n\n    def destroy_node(self):\n\n        if self.ser is not None and self.ser.is_open:\n            self.ser.close()\n            self.get_logger().info("UART fermé")\n\n        super().destroy_node()\n\n\ndef main(args=None):\n\n    rclpy.init(args=args)
```

```
node = BridgeDrone()

try:
    rclpy.spin(node)

except KeyboardInterrupt:
    pass

node.destroy_node()
rclpy.shutdown()

if __name__ == "__main__":
    main()
```


[3] Diagramme d'architecture globale matérielle et logicielle du système du Drone cible

